

CO3117 Machine Learning: Individual Longitudinal Assignment (two-part version)

- [1. Purpose, learning outcomes, and assessment philosophy](#)
 - [1.1 Alignment with the CO3117 syllabus and written final exam](#)
 - [1.2 Release-time adaptation for HK261](#)
- [2. Non-negotiable rule: one dataset, one use case](#)
 - [2.1 Canonical use case and approved alternatives](#)
- [3. Recommended study stack: three repositories, one MOOC, and the current textbooks](#)
 - [3.1 Recommended video MOOC: NPTEL Introduction to Machine Learning \(IIT Madras\)](#)
 - [3.2 Textbook/reference roles](#)
- [4. Required learning workflow for every major model family](#)
 - [4.1 Implementation-depth policy](#)
 - [4.2 Syllabus-to-assignment model matrix](#)
- [5. One repository, one wiki/blog-style knowledge base, and visible weekly progress](#)
 - [5.1 PROGRESS.md: instructor dashboard](#)
 - [5.2 Commit and checkpoint policy](#)
- [6. Weekly wiki/blog post: the living knowledge dossier](#)
- [7. Written-exam preparation track](#)
 - [7.1 Weekly first-attempt drill](#)
 - [7.2 Living two-A4 exam sheet](#)
 - [7.3 Final mock exam](#)
- [8. Two-part syllabus-aligned progress plan and minimum exercise prompts](#)
- [9. Common experimental protocol and per-model record](#)
 - [9.1 Required per-model record](#)
- [10. Two graded submission parts and instructor progress checks](#)
 - [10.1 Ownership checks](#)
- [11. Evaluation rubric \(100 points\)](#)
- [12. Academic integrity, repository use, and AI tools](#)
 - [12.1 Mandatory AI-assisted inquiry-based learning \(IBL\) protocol](#)
 - [12.2 Approved AI prompt patterns for inquiry, not answer generation](#)
 - [12.3 Prohibited or low-ownership uses of AI](#)
 - [12.4 Active video-learning protocol for the NPTEL lectures](#)
 - [12.5 Minimum AI USE.md record](#)
- [13. Final report: concise synthesis, not a dump of weekly posts](#)
- [14. Suggested topic-to-resource map](#)
- [15. References and course resources](#)
- [Appendix A. Interfaces of the Depth-A components](#)

CO3117 - MACHINE LEARNING

INDIVIDUAL LONGITUDINAL ASSIGNMENT — TWO-PART VERSION

ONE DATASET, ONE USE CASE, MANY MODELS

A two-part theory → code → experiment → written-exam learning portfolio

Design principle. Keep the dataset and prediction problem fixed. Every week, change the model or representation—not the problem—so that differences in assumptions, optimization, inductive bias, and inference can be compared meaningfully.

Item	Specification
Format	Individual; one continuous Git repository; one dataset; one use case; release-adapted checkpoints; TWO graded parts; W01–W02 catch-up; W08 midterm exception.
Workload	Approximately 45 hours total. From release: 2–3 hours for the one-time catch-up, about 3–4 hours in ordinary assignment weeks, and a lighter W08 midterm load.
Core evidence	Own implementation, code-reading log, controlled experiments, weekly wiki/blog learning trail, release baseline, written-exam drills, prospective Git history, Part I package, and Part II/final synthesis.
Submission parts	Part I: due 14 Oct 2026 (two days before midterm on 16 Oct); Part I syllabus material. Part II: due two calendar days before the official final exam; remaining syllabus material + final synthesis.
Exam-preparation track	Release-day baseline + weekly drills + Part I timed midterm preparation + post-midterm drills + 90-minute mock final; maintain a two-A4-page living exam sheet with Part I and Part II versions.
Study stack	CO3117 notes/textbooks → active NPTEL video lecture → own attempt → ML-From-Scratch → numpy-ml for HMM → pyprobml/Murphy for probabilistic depth → scikit-learn for benchmark.

1. Purpose, learning outcomes, and assessment philosophy

This assignment converts the former multi-model project into an individual, longitudinal learning system. Students repeatedly revisit the same prediction task with the model families in the CO3117 syllabus. The goal is not leaderboard performance; the goal is to connect mathematical formulation, algorithmic steps, implementation choices, empirical behavior, and written explanation.

- Explain core ML concepts, assumptions, objectives, update rules, inference procedures, and model limitations in precise written form.

- Implement selected core mechanisms from scratch in NumPy/Python and validate them against trusted implementations.
- Read reference code critically and map equations or algorithm steps to concrete code paths.
- Compare model families under a common split, preprocessing protocol, metric, and test population.
- Build a cumulative, inspectable learning record through Git commits, a wiki/blog-style knowledge base, weekly exercises, and two graded submission parts.
- Demonstrate individual ownership: the student must be able to derive, locate, explain, test, and modify any submitted component.

1.1 Alignment with the CO3117 syllabus and written final exam

The syllabus emphasizes understanding fundamental concepts, deriving the mathematical foundations and training mechanisms of representative models, comparing strengths and limitations, applying appropriate algorithms, performing preprocessing/dimensionality reduction, and evaluating models in realistic settings.

The sample 90-minute final exam (code 504, 26 May 2026) uses short constructed-response questions rather than programming. It tests conceptual definitions, comparisons, conditional independence in Bayesian networks, PCA, AdaBoost, generative versus discriminative modeling, and SVM reasoning. This assignment therefore contains a parallel written-exam track in addition to code and experiments.

Important. The assignment is designed to improve both practical ownership and written theoretical fluency. Code alone is not sufficient; prose alone is not sufficient.

1.2 Release-time adaptation for HK261

This specification takes effect in Course Week 3, corresponding to calendar week 38 (16 September 2026). The first class meeting was in calendar week 35; calendar week 36 (2 September) had no class because of the National Day holiday; calendar week 37 was the second teaching meeting. Therefore, no student is expected to fabricate W01–W02 Git history, back-date commits, or claim historical “pre-AI” attempts that did not exist.

- W01–W02 are treated as PRE-RELEASE content. Complete one concise PRE_RELEASE_CATCHUP.md plus one release-day handwritten baseline diagnostic covering the essential foundations and Decision Tree material. Date both artifacts honestly at the time they are created.
- Prospective ownership evidence starts now, in W03. The full two-state commit rule (first attempt → study/reference → corrected/extended state), weekly post, drill, and tag are mandatory from W03 onward.
- Course Week 8 is a protected MIDTERM week. The midterm is scheduled for 16 October 2026. Part I must already be frozen and submitted on 14 October 2026, exactly two calendar days earlier. Do not start a new major implementation or heavy experiment during the midterm window. After the exam, add only a short error/reflection note; this belongs to Part II learning evidence and must not rewrite Part I history.

- Catch-up work for W01–W02 is capped at approximately 2–3 hours total. It must not crowd out the current week’s learning. In ordinary non-midterm weeks after release, plan approximately 3–4 hours of assignment work.
- Release checkpoint R0 is due before the next scheduled CO3117 class (Week 4). After R0, weekly checkpoints continue as ownership evidence, but the assignment has only TWO graded submission parts: Part I is due 14 October 2026; Part II is due exactly two calendar days before the official final examination date. The LMS/instructor cutoff time always overrides any default time implied by this document.
- The timetable screenshot also contains a later no-class slot. To avoid ambiguity, all academic requirements below are indexed by COURSE WEEK (W03...W15), not by simple arithmetic on ISO calendar-week numbers.

2. Non-negotiable rule: one dataset, one use case

Core rule. The dataset source/version, prediction target, decision context, and held-out test population remain fixed for the entire semester. Different representations of the SAME raw data are allowed; changing to a second dataset merely to accommodate a model is not.

- Freeze the dataset source/version, target, split policy, primary metric, and random seeds by the end of the W03 release checkpoint (R0), before Week 4 begins.
- Fit scalers, encoders, PCA/LDA, and feature-selection procedures on training data only; then transform validation/test data.
- Keep the test set sealed for final comparison; tuning occurs only on training/validation data.
- For sequential/structured models, derive an ordered or structured view from the same raw dataset and document the transformation.
- If a model is genuinely unsuitable for the use case, submit a Model Suitability Study explaining the mismatch instead of forcing a scientifically meaningless experiment.

2.1 Canonical use case and approved alternatives

Unless an alternative is approved, the canonical dataset is the UCI Human Activity Recognition Using Smartphones family. The stable use case is: predict a person’s current physical activity from smartphone inertial measurements; for HMM/CRF, use temporal continuity to improve or smooth the same activity labels.

The instructor-provided prior-cohort report “Detect AI Generated Text” may be used as a STRUCTURAL EXEMPLAR for building a theory → implementation → experiment → reflection dossier. It is not an answer key. Students must not reuse its prose, code, tables, or reported results. If AI-generated-text detection is approved as the student’s use case, exactly one dataset/version must be frozen and any sequence-model reformulation must be justified explicitly.

3. Recommended study stack: three repositories, one MOOC, and the current textbooks

The resources below have different roles. None is authoritative by itself; students must cross-check equations, test implementations, and cite every source they actually use.

Resource	URL	Best role for CO3117	Required use pattern
ML-From-Scratch	https://github.com/eriklindernoren/ML-From-Scratch	Primary code microscope for classical ML: Decision Tree, Perceptron/MLP, Naive Bayes, GA, SVM, PCA/LDA, ensembles, logistic regression. Read only after committing an own first attempt for required Build tasks.	Do not copy code. Trace equations → functions → updates; record differences; make one meaningful modification where assigned.
numpy-ml	https://github.com/ddbourgin/numpy-ml	Primary HMM/sequence reference. It includes Viterbi, likelihood, forward-backward/Baum-Welch style routines and other legible NumPy implementations.	Implement at least one HMM core routine yourself first; then use numpy-ml to validate and extend.
pyprobml	https://github.com/probml/pyprobml	Theory-to-code bridge for Kevin Murphy's Probabilistic Machine Learning. Best for probabilistic formulation, figures, experiments, and deeper conceptual checks.	Use chapter/notebook-specific citations; never submit an unchanged notebook as your own work.

3.1 Recommended video MOOC: NPTEL Introduction to Machine Learning (IIT Madras)

Recommended single video-lecture companion: NPTEL/SWAYAM, Introduction to Machine Learning, Prof. Balaraman Ravindran, IIT Madras (<https://nptel.ac.in/courses/106106139>). This course is a particularly strong match to CO3117 because its weekly sequence includes bias–variance, logistic regression and LDA, perceptron and SVM, neural networks and backpropagation, MLE/MAP/Bayesian estimation, decision trees, evaluation and cross-validation, bagging/boosting, Naive Bayes, Bayesian networks, graphical models, and HMMs. Students should use the videos as active-learning material, not as passive viewing.

No single MOOC matches every CO3117 chapter. Use the NPTEL course as the primary video-lecture spine, then use the CO3117 notes, current textbooks, and the three repositories for gaps or deeper treatment, especially Genetic Algorithms, full PCA treatment, Maximum Entropy, CRF, and implementation-level study.

3.2 Textbook/reference roles

Course text/reference	Recommended role
Müller & Guido (2017), Introduction to Machine Learning with Python	Practical classical ML, preprocessing, model behavior, evaluation, scikit-learn workflows. Use as implementation/behavior companion, not as a substitute for derivations.
Chollet (2021), Deep Learning with Python, 2nd ed.	Neural-network intuition, training, backpropagation-related practice, and modern deep-learning context.
Murphy (2022), Probabilistic Machine Learning: An Introduction	Primary theory reference for probability, Bayesian/generative-discriminative views, graphical/sequential models, probabilistic inference, and many mathematical foundations.
Tom Mitchell (1997), Machine Learning	Classic conceptual treatment of learning, decision trees, Bayesian learning, hypothesis spaces, and foundational terminology.
Stephen Marsland (2009), Machine Learning: An Algorithmic Perspective	Algorithm-oriented implementation perspective; useful when converting mathematics/pseudocode into code and for evolutionary/optimization viewpoints.
Cao Hoàng Trụ (2008), Trí tuệ Nhân tạo = Thông minh + Giải thuật	Vietnamese-language conceptual/algorithmic context and complementary AI perspective.
Reference discipline. A repository, MOOC, blog, textbook, AI tool, or library can help you learn; none can replace your own first attempt, your own explanation, and your ability to answer a written question without code.	

4. Required learning workflow for every major model family

Stage	Minimum evidence
1. UNDERSTAND	Read CO3117 notes + selected textbook/MOOC material. Write the model, assumptions, objective/factorization, learning or inference rule, and expected failure modes in your own words.
2. FIRST ATTEMPT	Solve the week's written drill and, where required, implement the core mechanism before consulting repository code. Commit this state.
3. DISSECT	Inspect the relevant ML-From-Scratch / numpy-ml / pyprobml code or notebook. Map at least three mathematical/algorithmic steps to exact code locations.

Stage	Minimum evidence
4. MODIFY / COMPLETE	Fix your implementation or make the assigned non-trivial modification. Do not erase the first attempt; preserve history.
5. BENCHMARK	Compare with scikit-learn or another approved implementation under the same split and preprocessing.
6. EXPLAIN	Update the weekly wiki/blog post, error analysis, MODEL_LOG, and the two-A4 exam sheet. Commit and tag the weekly checkpoint.

4.1 Implementation-depth policy

Depth	Meaning	Minimum evidence
A — BUILD	Implement the required core algorithm yourself with NumPy/Python; no library estimator for the required mechanism.	Derivation/pseudocode; own code; sanity/unit tests; trusted benchmark; pre-reference and post-reference commits.
B — DISSECT & MODIFY	Study a reference implementation, explain it, and make a non-trivial modification/adaptation.	Exact attribution; code-reading notes; meaningful diff; experiment demonstrating the effect of the modification.
C — APPLY & BENCHMARK	Use an approved implementation to study applicability, tuning, behavior, and limitations.	Correct formulation; justified hyperparameters; controlled benchmark; error/limitation analysis.

4.2 Syllabus-to-assignment model matrix

Syllabus	Model family	Depth	Required artifact
Ch. 1	Foundations; metrics; over/underfitting; bias-variance	Analysis	Learning/validation curve, leakage check, model taxonomy, exam-style conceptual answers.

Syllabus	Model family	Depth	Required artifact
Ch. 2	Decision Tree	B (release catch-up)	Release catch-up: implement one impurity/split routine yourself; dissect a complete tree implementation; analyze continuous/missing attributes and pruning/stopping; benchmark. A full tree from scratch is optional enrichment, not retroactive mandatory work.
Ch. 3	Perceptron / MLP / backpropagation	A	Explicit forward/backward pass; gradient/numerical sanity check; capacity/regularization experiment.
Ch. 4	Bayesian / Naive Bayes	A	Own Naive Bayes appropriate to representation; smoothing/variance handling; assumption analysis.
Ch. 5	Genetic Algorithm	B	Student-authored representation, fitness, selection, crossover, mutation; feature-selection or bounded search experiment.
Ch. 6	Bayesian network / TAN / HMM	HMM A(core)+B; BN/TAN C/theory	Own Forward or Viterbi core; numpy-ml validation; BN factorization and d-separation exercises.
Ch. 7	SVM; kernels; soft margin	B+C	Dissect margin/hinge-loss logic; linear vs kernel; effect of C/scaling; compare with logistic/perceptron.

Syllabus	Model family	Depth	Required artifact
Ch. 8	PCA / LDA / feature reduction	PCA A; LDA C	Own PCA; explained variance/reconstruction checks; compare PCA vs LDA and classifier behavior.
Ch. 9	Bagging / boosting / ensembles	B+C	Bagging and boosting study; AdaBoost weight-update logic; bias/variance and error analysis.
Ch. 10	Generative vs discriminative; Logistic/MaxEnt; CRF	Logistic/softmax A; CRF C	Own logistic/softmax; MaxEnt interpretation; CRF vs HMM/model-suitability analysis; final synthesis.

5. One repository, one wiki/blog-style knowledge base, and visible weekly progress

All work must live in one continuous repository. The wiki/blog content should preferably be stored as Markdown in the same repository so that its evolution is visible in Git history. Keep the dossier as Markdown under docs/ and read it on GitHub. Do not enable GitHub Pages or a GitHub Wiki: a Pages site built from a private repository is public on the Internet. The repository is private and is issued by the instructor in the course organisation HCMUT-CO3117-261 under the name ml-<student ID>. Its README describes the weekly routine and the self-check tools/check.py, which also runs the contract tests of Appendix A. Deadlines are judged by the time GitHub received a checkpoint tag, not by the dates recorded in commits.

```
co3117-ml-individual/
├── README.md
├── PROGRESS.md
├── MODEL_LOG.md
├── AI_USE.md
├── REFERENCES.md
├── SUBMISSION_PART1.md
├── SUBMISSION_PART2.md
├── environment.yml / requirements.txt / pyproject.toml
├── data/README.md
├── docs/
│   ├── index.md
│   ├── pre-release/
│   │   └── PRE_RELEASE_CATCHUP.md
│   └── weekly/
├── w03-perceptron-delta.md
└── w04-ann-backprop.md
```

```

|   └── ... w15-synthesis.md
├── exercises/
|   ├── release-baseline-w01-w02.pdf
|   ├── w03-first-attempt.pdf
|   ├── w03-corrections.md
|   └── ...
├── exam/
|   ├── a4-notes-part1-midterm.pdf
|   ├── midterm-reflection.md
|   ├── a4-notes-part2-final.pdf
|   ├── mock-final-first-attempt.pdf
|   └── mock-final-corrections.md
├── src/
|   ├── data.py
|   ├── metrics.py
|   ├── from_scratch/
|   └── reference_adapters/
├── experiments/
|   ├── part1_pre_midterm/
|   └── part2_post_midterm/
├── tests/
├── results/
|   ├── metrics.csv
|   └── figures/
├── report/
├── part1_summary.pdf
└── part2_final_report.pdf

```

5.1 PROGRESS.md: instructor dashboard

PROGRESS.md must contain one row per Course Week. Mark W01–W02 explicitly as PRE-RELEASE and link them to the single catch-up/baseline package; mark W08 as MIDTERM. For W03–W07 and W09–W15, link the week’s theory post, code/experiment, written drill, first-attempt commit, post-reference commit, and tag. The instructor should be able to verify progress in less than two minutes without searching the repository.

Period	Topic	Post	Drill	First evidence	Revision commit	Tag	Status
W01–W02	PRE-RELEASE catch-up	combined post	release baseline	current-date baseline	catch-up corrections	release-baseline	PRE-RELEASE
W03	Perceptron / Delta + onboarding	wiki/blog	exercise	pre-ref / pre-AI	post-ref	w03	ACTIVE

5.2 Commit and checkpoint policy

- From W03 onward, each ordinary assignment week must preserve at least two substantive states: (i) the student's first attempt before reference-code inspection/AI assistance and (ii) the corrected/extended state after study, testing, and benchmarking. W01–W02 use the release-day baseline exception described above; W08 uses the midterm exception.
- Expected commit-message pattern: [W03][baseline] release diagnostic and setup; [W07][theory] BN d-separation notes; [W07][code] own factorization checks; [W07][review] corrected after Murphy/pyprobm1.
- Create tag release-baseline at R0, then lightweight tags w03, w04, ..., w15 after each Course Week checkpoint (use w08-midterm for the midterm artifact). Create exactly two graded-submission tags: part1-final at the 14 October 2026 deadline and part2-final at the Part II deadline. Do not create fake w01/w02 historical tags.
- After a weekly tag has been checked, do not rewrite history by force-pushing/rebasing that checkpoint without instructor approval. Late work should remain visibly late rather than being back-dated or history-rewritten.
- Commit quality matters more than commit count. Artificial micro-commits do not create ownership evidence; a single end-of-semester code dump does not satisfy longitudinal progress.

6. Weekly wiki/blog post: the living knowledge dossier

The prior-cohort “Detect AI Generated Text” report illustrates a valuable pattern: learn theory, implement/apply it, analyze evidence, and reflect. CO3117 adopts that pattern in a concise form. Submit one combined 500–900 word PRE-RELEASE catch-up post for W01–W02, then one normal 350–700 word post for each ordinary week W03–W07 and W09–W15. W08 requires only a compact midterm-preparation/reflection entry. Do not write a 400-page report.

Required subsection	Content
A. Concept capsule	Definitions, assumptions, model class, objective/factorization, training/inference rule.
B. One derivation or worked example	A derivation, small numerical calculation, d-separation argument, PCA step, AdaBoost update, margin calculation, etc.
C. Code-to-theory trace	Link 2–4 equations/algorithm steps to exact lines/functions in your own code; after reference inspection, add the external code location too.
D. One controlled experiment	A single focused question: tree depth, smoothing, network capacity, C/kernel, number of PCs, number of estimators, sequence smoothing, etc.
E. Failure / misconception	One failure case, counterexample, limitation, or common misconception.

Required subsection	Content
F. Written-exam capsule	Your final 4–8 sentence explanation of the week's most examinable concept, written without code.
G. Reflection	What you can now explain that you could not explain before; what remains uncertain; what you will test next.
H. Inquiry trail (mandatory whenever AI is used)	State the question you asked, link the pre-AI attempt commit, summarize the AI hint/question (not a pasted solution), identify the textbook/MOOC/repository source used to verify it, and write what you can now reproduce without AI.
Ownership signal. The post must point to actual commits, tests, figures, and experiments. Generic textbook-like prose with no connection to the student's repository is weak evidence of learning.	

7. Written-exam preparation track

The assignment includes an explicit no-code written-practice stream because the sample final exam is a 90-minute constructed-response paper. Coding competence does not automatically transfer to concise theoretical answers.

7.1 Weekly first-attempt drill

- For W03–W07 and W09–W15, complete one 15–25 minute handwritten drill (paper or stylus; accessibility accommodations permitted) with no code, no Internet search, no repository, and no AI. At release, complete one additional baseline diagnostic covering W01–W02. In W08, replace the normal drill with a timed midterm rehearsal appropriate to the instructor's exam format.
- Upload/commit the first attempt BEFORE checking notes or solutions. The scan/photo/PDF is part of the ownership evidence. This prospective rule starts at the W03 release; do not manufacture historical first attempts for W01–W02.
- Limitation, stated openly: a committed scan proves when the file reached GitHub, not that no note, solution or AI tool was consulted before it was written. This rule therefore rests on honesty and on the ownership checks of Section 10.1, and the instructor may hold some drills in class instead.
- Afterward, check the course notes/textbooks/MOOC, write corrections in Markdown, cite the source, and commit a corrected version. Never replace or delete the first attempt.
- Each drill should contain four short subquestions in the style of the sample final, plus a small derivation/numerical/graphical item when appropriate.
- AI may be used only after the first-attempt commit for explanation/checking under the course AI policy, and such use must be recorded in AI_USE.md.

7.2 Living two-A4 exam sheet

Because the sample final exam permits two A4 sheets, begin the two-page “living exam sheet” at release. It should contain definitions, canonical equations, algorithm steps, comparison tables, and tiny worked examples—not copied prose. Submit a Part I version on 14 October 2026 for midterm preparation and a Part II/final version two calendar days before the final exam. The compression process is itself a learning exercise.

7.3 Final mock exam

In Week 15, complete one 90-minute mock final under the same self-imposed conditions as the sample: no code/Internet/AI, only the current two-A4 sheet. Commit the untouched first attempt, then a separate self-marking/correction file that identifies knowledge gaps and links each correction to the relevant weekly post or source.

8. Two-part syllabus-aligned progress plan and minimum exercise prompts

The 15-week syllabus is divided into TWO assignment parts. Part I covers all material taught before the midterm boundary: Foundations/ML workflow (Chapter 1), Decision Trees (Chapter 2), Artificial Neural Networks (Chapter 3), Bayesian learning / Naive Bayes (Chapter 4), Genetic Algorithms (Chapter 5), and the Part I portion of Chapter 6 on graphical models/Bayesian Networks as actually covered in class. Part I is due 14 October 2026, two calendar days before the midterm on 16 October 2026. Part II starts after the Part I freeze and covers the remaining Chapter 6 material (especially HMM), Chapter 7 SVM, Chapter 8 dimensionality reduction, Chapter 9 ensemble learning, Chapter 10 discriminative models, plus final cross-model synthesis. Part II is due exactly two calendar days before the official final exam date. Weekly work remains visible throughout; only the formal graded packaging is split into these two parts.

Wk	Topic	Repository / experiment evidence	Minimum written drill
1 (pre-release; cal. 35)	Foundations: ML workflow, metrics, over/underfitting, bias–variance	RETROSPECTIVE ONLY. Put the baseline pipeline, leakage checklist, model taxonomy, and one train/validation-curve diagnosis in the catch-up post. No back-dated Git history.	Covered in the release-day W01–W02 baseline diagnostic: definitions, under/overfitting, bias–variance, and curve diagnosis.
2 (pre-release; cal. 37)	Decision Tree (calendar week 36 holiday occurred between W1 and W2)	RETROSPECTIVE ONLY. Implement one impurity/split routine; inspect a full tree; add one stopping/pruning comparison. A full from-scratch tree is optional enrichment.	Covered in the release-day baseline: impurity/information gain, continuous attributes, missing values, pruning/generalization.

Wk	Topic	Repository / experiment evidence	Minimum written drill
3 — RELEASE (cal. 38)	Perceptron and Delta rule + repository onboarding	Create repository structure, PROGRESS.md, AI_USE.md, dataset/use-case draft, release-baseline tag, and catch-up. Implement own perceptron and connect the update to geometry. Freeze data protocol by end of W03.	(A) Perceptron update. (B) Linear separability. (C) Why XOR fails. (D) Perceptron vs delta/gradient learning.
4	Artificial Neural Networks and backpropagation	Small MLP; explicit forward/backward pass; numerical gradient/sanity check; capacity experiment; first normal weekly two-state commit trail.	(A) Role of non-linearity. (B) Forward pass. (C) Chain rule/backprop. (D) Learning-rate/capacity/regularization effects.
5	Bayesian learning / Naive Bayes	Own NB; smoothing/variance handling; assumption-violation analysis; common protocol benchmark.	(A) Bayes rule. (B) Naive conditional-independence assumption. (C) Smoothing. (D) Why NB is generative.
6	Genetic Algorithm	Feature/subset chromosome; student-authored fitness, selection, crossover, mutation; bounded search; convergence plot.	(A) Representation. (B) Fitness. (C) Selection/crossover/mutation. (D) Premature convergence and exploration–exploitation.
7 — PART I CLOSEOUT / PRE-MIDTERM	Bayesian Networks / TAN	Small BN/TAN factorization; d-separation examples; conditional-independence note; consolidate all Part I evidence. Freeze the common protocol, update SUBMISSION_PART1.md, produce part1_summary.pdf and a4-notes-part1-midterm.pdf, and create tag part1-final by 14 Oct 2026.	(A) Factorize a BN. (B) Chain/fork/collider. (C) Conditional independence under evidence. (D) NB vs TAN.

Wk	Topic	Repository / experiment evidence	Minimum written drill
8 — MIDTERM (16 Oct 2026)	Protected written-exam week	NO NEW MAJOR MODEL IMPLEMENTATION. Part I is already frozen. Do a timed midterm rehearsal before the exam; after the exam add a concise error/reflection note within 72 hours as Part II evidence. Do not rewrite Part I.	Use instructor/sample-exam style. Focus on concise definitions, derivations, comparisons, and conditional-independence reasoning. Corrections occur only after the timed attempt.
9	Hidden Markov Model / sequence modeling	Own Forward or Viterbi core; tiny synthetic test with known answer; use numpy-ml for fuller HMM; compare independent vs sequence-aware predictions.	(A) HMM components/factorization. (B) Forward recursion. (C) Viterbi vs Forward. (D) Filtering/decoding and i.i.d. vs sequence assumptions.
10	SVM: maximum margin and soft margin	Dissect SVM; linear SVM; vary C; scaling experiment; compare objective with logistic/perceptron.	(A) SVM vs logistic objective. (B) Why maximum margin. (C) Support vectors. (D) Meaning/effect of C.
11	Kernel SVM and cross-model comparisons	Linear vs RBF/kernel experiment; compare SVM with perceptron/logistic/MLP; predict effects before running.	(A) Kernel trick. (B) Linear vs RBF. (C) Nonlinear data vs neural networks. (D) SVM advantage over simple perceptron.
12	PCA and curse of dimensionality	Own PCA; eigensystem/SVD check; explained variance; reconstruction; classifier-after-PCA; continue Part II cumulative comparison.	(A) Curse of dimensionality + consequences. (B) Two goals of PCA. (C) Meaning of variance. (D) Decorrelation of principal components.
13	LDA, feature engineering, dimensionality reduction	Compare PCA vs LDA; choose dimension; class-separation visualization/analysis; controlled feature experiment.	(A) PCA vs LDA objective. (B) Supervised vs unsupervised reduction. (C) Maximum useful LDA dimension. (D) When reduction can hurt.

Wk	Topic	Repository / experiment evidence	Minimum written drill
14	Bagging, Boosting, AdaBoost	Bagging + boosting benchmark; error analysis; trace AdaBoost sample reweighting and weighted vote.	(A) “AdaBoost” meaning. (B) Boosting vs bagging training. (C) Sample-weight update. (D) Final prediction combination.
15 — FINAL COURSE WEEK / PART II SYNTHESIS	Generative vs discriminative; Logistic/MaxEnt; CRF; synthesis	Own logistic/softmax; MaxEnt connection; CRF/HMM suitability study; final comparison; 90-min mock final; complete Part II final report and final two-A4 sheet. Formal Part II submission/tag part2-final is due exactly two calendar days before the official final exam date.	(A) Generative vs discriminative. (B) Examples/joint vs conditional. (C) Why discriminative may be preferred. (D) CRF vs HMM; cumulative exam-style synthesis.

9. Common experimental protocol and per-model record

Protocol item	Requirement
Primary metric	Macro-F1 for multiclass classification unless another metric is instructor-approved. Always report accuracy and a confusion matrix as secondary evidence.
Split policy	Use group/subject-aware splitting when repeated measurements come from the same person/entity. Prevent entity leakage across train and test.
Validation	Tune on validation data or cross-validation inside the training population only. Keep the final test population sealed.
Preprocessing	Fit all preprocessing and representation-learning steps on training data only.
Baselines	Include at least one simple baseline and preserve it throughout the semester.
Hyperparameters	Use small, justified searches. Report the search space and selection criterion.
Runtime	Record environment and measure training/inference consistently; focus on interpretable relative comparisons.
Complexity	State expected time/space behavior for core mechanisms and relate it to observed behavior.

Protocol item	Requirement
Failure analysis	Inspect at least five representative errors or one systematic confusion/failure mode per major model family.
Statistical caution	Do not claim superiority from a tiny metric difference without repeat runs, confidence analysis, or other defensible evidence.

9.1 Required per-model record

- Model name, syllabus chapter, and implementation depth (A/B/C).
- Mathematical objective / probabilistic factorization / decision rule as applicable.
- Assumptions, inductive bias, and expected failure modes.
- Data representation and preprocessing used.
- Own pre-reference commit and exact reference code/notebook consulted.
- Hyperparameters and selection procedure.
- Primary/secondary metrics, runtime, and feature/model size when meaningful.
- One diagnostic plot/table and one focused experiment.
- Error/limitation analysis and use-case fit.
- One exam-ready paragraph: explain the core idea without code.

10. Two graded submission parts and instructor progress checks

Checkpoint / part	Required evidence and deadline	Assignment points
R0 — setup gate (not a graded part)	Repository skeleton; PROGRESS.md; AI_USE.md; honest W01–W02 catch-up; release-day baseline diagnostic; dataset/use-case draft; frozen split/metric/seeds; tag release-baseline. Due before W04 class.	Gate only

Checkpoint / part	Required evidence and deadline	Assignment points
PART I — PRE-MIDTERM PORTFOLIO	DUE 14 OCTOBER 2026 — two calendar days before the midterm on 16 October. Include W01–W02 catch-up plus prospective W03–W07 evidence; Decision Tree catch-up; Perceptron/MLP; Bayesian/Naive Bayes; Genetic Algorithm; Part I Chapter 6 Bayesian Network/TAN/d-separation material as actually taught; stable common experimental protocol; weekly wiki/blog posts; written drills; SUBMISSION_PART1.md; part1_summary.pdf; a4-notes-part1-midterm.pdf; tag part1-final. HMM belongs to Part II unless the instructor explicitly assigns it before the Part I boundary.	40 / 100
PART II — POST-MIDTERM + FINAL PORTFOLIO	DUE EXACTLY TWO CALENDAR DAYS BEFORE THE OFFICIAL FINAL EXAM DATE. Include post-Part-I weekly evidence; remaining Chapter 6 (especially HMM); SVM/soft-margin/kernels; PCA/LDA and feature engineering; bagging/boosting; generative vs discriminative modeling; logistic/MaxEnt; CRF or justified suitability study; cumulative model comparison; error/limitation analysis; 90-minute mock final; SUBMISSION_PART2.md; part2_final_report.pdf; a4-notes-part2-final.pdf; tag part2-final.	60 / 100

Submission freeze rule: once Part I is submitted/tagged on 14 October 2026, its commits and files are preserved as historical evidence. Any correction discovered after the midterm must be recorded in Part II (for example, PART1_ERRATA.md or the midterm reflection) rather than by rewriting, rebasing, or replacing the Part I history. The same rule applies after Part II is tagged.

10.1 Ownership checks

At any checkpoint the instructor may select a random artifact and ask the student to perform one or more of the following without preparation. Ownership checks apply to work created after release; W01–W02 are judged through the current catch-up/baseline, not imaginary historical Git evidence:

- Derive or explain the equation/algorithm used in a submitted model.

- Locate the corresponding function/line in the student's code and in the cited reference implementation.
- Explain one surprising result or failure case using evidence from the experiment.
- Change a small parameter or code fragment live and predict the effect before running it.
- Answer one exam-style short question orally or on paper.
- Explain the difference between the first-attempt and post-reference commits.

Ownership standard. A student is expected to understand every submitted component. "It runs" is not evidence of ownership; the student must connect theory, code, and results.

11. Evaluation rubric (100 points)

Criterion	Pts (P1+P2)	Full-credit evidence
1. Weekly progress & Git ownership	12 (5+7)	Honest release baseline + continuous checkpoints; pre/post-reference history in ordinary weeks; w08-midterm exception; part1-final and part2-final tags; meaningful progression; no backdating or deadline dump.
2. Weekly wiki/blog knowledge dossier	13 (6+7)	One concise W01–W02 catch-up plus accurate W03–W07 and W09–W15 theory posts; derivations/examples, code-to-theory traces, focused experiments, reflections, citations; compact W08 midterm entry.
3. Written-exam portfolio	15 (8+7)	Release baseline; weekly handwritten first attempts; Part I midterm-ready two-A4 sheet; post-midterm corrections; Part II final two-A4 sheet; 90-min final mock; evidence of improved concise explanations.
4. Required from-scratch implementations	18 (8+10)	Depth-A components are student-authored, correct, tested, numerically/synthetically validated.
5. Reference-repository dissection & modification	10 (4+6)	Exact sources, MODEL_LOG, code mapping, meaningful modifications, no disguised copying.

Criterion	Pts (P1+P2)	Full-credit evidence
6. Experimental design & trusted benchmarks	12 (4+8)	Fixed data protocol, fair comparisons, justified tuning, sealed test set, appropriate baselines.
7. Evaluation, error analysis & limitations	10 (3+7)	Metrics interpreted; failures investigated; model suitability and trade-offs justified with evidence.
8. Reproducibility & software engineering	5 (1+4)	Environment, seeds, setup/run commands, tests, clean repository, reproducible results.
9. Final synthesis/report & resource discipline	5 (1+4)	Part I summary plus Part II cross-model synthesis/final report; concise writing; proper citations; AI/repository use disclosed; clear linkage to exam-ready understanding.

The assignment score is converted to the course assignment weight specified in the CO3117 syllabus. For this two-part version, Part I contributes 40 points and Part II contributes 60 points to the 100-point assignment score. The criterion rubric below shows the Part I + Part II allocation for each criterion. No student is penalized for the assignment not existing in W01–W02; however, from the release onward, weekly progress evidence cannot be reconstructed retrospectively by uploading finished work at a deadline.

12. Academic integrity, repository use, and AI tools

The CO3117 syllabus permits AI tools only for supportive purposes and requires disclosure. The same principle applies to external repositories, MOOC solutions, code assistants, and online notebooks.

- Depth-A code must be written by the student. Reference-code inspection occurs only after a first attempt/core routine has been committed.
- Depth-B adaptation requires exact attribution and a meaningful student modification; copying followed by cosmetic renaming is not acceptable.
- The first-attempt weekly written drill must be completed without AI or online solution lookup.
- AI may be used afterward for explanations, debugging ideas, test generation, grammar/presentation support, or reference discovery, subject to the course policy.
- AI_USE.md must record the tool, purpose, affected files/components, and what the student independently checked.
- The student remains responsible for correctness, licensing, citations, reproducibility, and the ability to explain every submitted artifact.

12.1 Mandatory AI-assisted inquiry-based learning (IBL) protocol

AI may support learning only when it increases the student’s questioning, prediction, checking, and reconstruction. The preferred role is Socratic tutor, verifier, adversarial questioner, and test generator—not ghostwriter or solution engine. Productive struggle must occur before AI assistance. Every AI-assisted learning episode must leave evidence that the student can later reproduce the reasoning without the tool.

Phase	Required behavior	Ownership evidence
0. FRAME	Write a concrete learning question and your current belief/guess before opening AI.	Weekly post: “Question I am trying to answer” + 2–5 sentence initial hypothesis.
1. ATTEMPT	Solve/derive/code first without AI. Commit the attempt even if incomplete or wrong.	W03 onward: pre-AI commit hash plus handwritten drill or own code/derivation. W01–W02: use the current release-day baseline; never fabricate historical evidence.
2. INQUIRE	Ask AI for one Socratic question, hint, counterexample, or diagnostic at a time. Do not request a final solution.	AI_USE.md entry with purpose and concise summary of the hint/question.
3. VERIFY	Check every important claim against CO3117 notes, textbook, NPTEL lecture, or cited repository/source.	Citation/link plus a note identifying what was confirmed, corrected, or left uncertain.
4. RECONSTRUCT	Close the AI response and rewrite the explanation, derivation, or code logic from memory in your own structure.	Post-AI commit must not be a pasted or lightly paraphrased AI answer.
5. TRANSFER	Solve a changed example, edge case, counterexample, or parameter variation that was not shown in the original explanation.	One new worked example/test demonstrating transfer rather than imitation.
6. TEACH-BACK	Explain the concept aloud or in writing as if teaching a peer; then let AI challenge assumptions or ask follow-ups.	A 4–8 sentence exam-ready capsule or short oral-check note.
7. DELAYED RETRIEVAL	Within 24–72 hours, answer one short question or reproduce one key derivation with no AI and no notes.	A small retrieval note/scan. If it fails, record the misconception and repeat.

Release exception: the mandatory pre-AI Git evidence applies prospectively from W03. For W01–W02, the release-day handwritten baseline is the honest “before assistance” evidence. Students must not back-date commits or invent earlier AI-use histories.

12.2 Approved AI prompt patterns for inquiry, not answer generation

Students may adapt prompts such as the following. The key rule is that the AI should make the student think, predict, justify, or debug before revealing information.

- “Do not solve this problem. Ask me one question at a time that helps me discover the answer. Wait for my response before continuing.”
- “Here is my derivation. Do not rewrite it. Identify only the first step that is unjustified or wrong, and ask me to repair it.”
- “I think this claim is true: [...]. Ask me for the assumptions, then give the smallest counterexample only if my claim is false.”
- “Give me a tiny dataset and ask me to perform exactly one iteration/update of this algorithm by hand. Do not show the answer until I commit mine.”
- “Quiz me in CO3117 final-exam style with four short subquestions on [topic]. Ask one at a time and score/explain only after I answer.”
- “Act as an oral examiner. Make me compare [model A] and [model B] in objective, assumptions, training, failure modes, and use-case fit. Push back on vague answers.”

12.3 Prohibited or low-ownership uses of AI

- Using AI to produce the first-attempt written drill, first derivation, or first Depth-A implementation.
- Asking AI to write the weekly wiki/blog post, final report section, reflection, or “exam-ready” answer and then submitting it with cosmetic edits.
- Copying AI-generated code into a component claimed as student-authored, even if the code runs correctly.
- Using AI to summarize a lecture/textbook that the student has not attempted to read/watch, then treating that summary as the source of record.
- Deleting failed attempts or replacing history so that the learning path appears cleaner than it actually was.
- Accepting an AI explanation without verifying equations, assumptions, or implementation behavior against an authoritative course source.

12.4 Active video-learning protocol for the NPTEL lectures

Watching a lecture is not, by itself, progress evidence. For each assigned video block, use a 3–2–1 active-viewing routine: (3) before watching, write three questions or predictions; (2) while watching, pause at least twice to derive/predict the next step before the lecturer explains it; (1) after watching, close the video/notes and write one compact concept map or 5-minute summary from memory. Then solve at least one exam-style question or small worked example and link it from the weekly post.

12.5 Minimum AI_USE.md record

Field	Required entry
Week/date	W__ / YYYY-MM-DD
Learning question	The precise concept/problem being investigated
Pre-AI evidence	W03 onward: commit hash or handwritten first-attempt artifact. For W01–W02 catch-up: cite the release-day baseline artifact.
AI tool	Tool/model used
Prompt purpose	Socratic hint, counterexample, debugging question, quiz, etc.
Hint/question received	Concise summary; do not paste a full generated solution
Verification source	CO3117 note, textbook section, NPTEL lecture, repository/code reference
What changed	Specific misconception, derivation, test, or code decision corrected
Closed-book reproduction	Yes / Not yet; link the delayed-retrieval artifact when available

The instructor may use AI_USE.md to select ownership-check questions. A student who cannot reproduce or defend an AI-assisted artifact may receive no credit for the affected ownership evidence even if the artifact itself is technically correct.

13. Final report: concise synthesis, not a dump of weekly posts

Section	Suggested length	Content
1. Problem & dataset	0.5 page	Use case, target, dataset/version, split, primary metric, leakage controls.
2. Representations & protocol	0.5 page	Static/sequential views, preprocessing, tuning protocol, runtime environment.
3. Model-family map	1–1.5 pages	Compact formulations and implementation depths; point to code/wiki rather than reproducing them.
4. Results	1–1.5 pages	One master comparison table plus only the most informative plots.

Section	Suggested length	Content
5. Error/behavior analysis	1 page	Systematic confusions/failures; effects of temporal structure, representation, and inductive bias.
6. Cross-model synthesis	1 page	Generative vs discriminative; parametric/non-parametric; bias/variance; margin/probability; sequence vs independent.
7. Ownership & learning reflection	0.5 page	What changed from first attempts; most important misconceptions corrected; what remains uncertain.
8. Reproducibility & references	0.5 page + refs	Run commands, environment, repository/MOOC/textbook citations, AI disclosure pointer.

Suggested final report length: approximately 6–8 pages excluding references and appendices. The PRE-RELEASE catch-up plus W03–W15 weekly artifacts may be exported separately as a learning portfolio but should not be pasted wholesale into the final report.

14. Suggested topic-to-resource map

Topic	Primary companions	How to use them
Foundations; overfitting; metrics	CO3117 notes; Müller & Guido; Murphy; NPTEL Weeks 0–1, 7	Focus on definitions, model selection, representation, and validation reasoning.
Decision Trees	Mitchell; Müller & Guido; Marsland; ML-From-Scratch; NPTEL Week 6	Derive split logic; then inspect implementation and pruning/stopping choices.
Perceptron / ANN / backprop	Chollet; Mitchell/Marsland; ML-From-Scratch; NPTEL Weeks 4–5	Be able to write updates/chain-rule reasoning by hand.
Bayes / Naive Bayes	Murphy; Mitchell; ML-From-Scratch; NPTEL Weeks 5 and 8	Emphasize joint/conditional distributions and assumptions.
Genetic Algorithm	Marsland; ML-From-Scratch; course notes (not a core NPTEL topic)	Treat representation, fitness, selection, crossover, mutation as algorithm-design choices.
Bayesian networks / HMM	Murphy; pyprobml; numpy-ml; NPTEL Weeks 8–9	Factorization, conditional independence, dynamic programming, likelihood/inference.
SVM / kernels	Müller & Guido; Marsland; ML-From-Scratch; NPTEL Week 4	Know objective/margin/support-vector ideas and comparisons with logistic/perceptron/NN.

Topic	Primary companions	How to use them
PCA / LDA	Murphy; Müller & Guido; ML-From-Scratch; NPTEL Weeks 2–3 (PCR/LDA) plus CO3117 PCA notes	Be able to explain variance, decorrelation, dimensionality, and supervised vs unsupervised reduction.
Bagging / Boosting	Müller & Guido; Mitchell/Marsland; ML-From-Scratch; NPTEL Weeks 7–8	Understand sequential reweighting vs independent bagging and weighted combination.
Logistic / MaxEnt / CRF	Murphy; pyprobml; course notes; NPTEL Week 3 for logistic and Week 9 for graphical-model context	Connect conditional modeling to discriminative learning; contrast CRF with HMM.

15. References and course resources

- [1] CO3117 Machine Learning course syllabus, HK261, Faculty of Computer Science and Engineering, HCMUT, VNU-HCM.
- [2] Sample CO3117 final examination, question sheet code 504, Semester 2 2025–2026, 26 May 2026.
- [3] Andreas C. Müller and Sarah Guido, Introduction to Machine Learning with Python, O'Reilly, 2017.
- [4] François Chollet, Deep Learning with Python, 2nd ed., Manning, 2021.
- [5] Kevin P. Murphy, Probabilistic Machine Learning: An Introduction, MIT Press, 2022.
- [6] Cao Hoàng Trụ, Trí tuệ Nhân tạo = Thông minh + Giải thuật, NXB ĐHQG-HCM, 2008.
- [7] Tom M. Mitchell, Machine Learning, McGraw-Hill, 1997.
- [8] Stephen Marsland, Machine Learning: An Algorithmic Perspective, Chapman & Hall/CRC, 2009.
- [9] Erik Linder-Norén, ML-From-Scratch, <https://github.com/eriklindernoren/ML-From-Scratch>.
- [10] David Bourgin, numpy-ml, <https://github.com/ddbourgin/numpy-ml>.
- [11] ProbML, pyprobml, <https://github.com/probml/pyprobml>.
- [12] Balaraman Ravindran, Introduction to Machine Learning, NPTEL/SWAYAM, IIT Madras, <https://nptel.ac.in/courses/106106139>.
- [13] scikit-learn documentation, <https://scikit-learn.org/stable/>.
- [14] UCI Machine Learning Repository, Human Activity Recognition Using Smartphones, <https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>.
- [15] Instructor-provided prior-cohort exemplar: CO3101 PIP, Detect AI Generated Text (2024).

Appendix A. Interfaces of the Depth-A components

Each Depth-A component in Section 4.2 has a fixed file and interface so that it can be checked automatically. The contract tests in tests/contract call these functions on small synthetic problems whose answers are known by another route, such as finite differences, exhaustive enumeration or a trusted library. A component becomes required from the week in the second column; before that week an unfinished stub is skipped, from that week on it fails. The tests run in GitHub Actions when a checkpoint tag is pushed, and locally with python tools/check.py.

Component	Required from	File	Interface	What the contract test checks
Impurity / split routine (Ch. 2, catch-up)	W03	src/from_scratch/tree_split.py	entropy(y), gini(y), information_gain(y, mask), best_threshold(x, y) -> (t, gain)	Known values; gain of perfect and useless splits; best threshold equal to brute force over all midpoints; constant feature returns (None, 0.0)
Perceptron (Ch. 3)	W03	src/from_scratch/perceptron.py	perceptron_update(w, b, x, y, lr) -> (w, b); class Perceptron: fit(X, y), predict(X), labels in {-1, +1}	Update only on a mistake; convergence on linearly separable data with a margin
MLP with backpropagation (Ch. 3)	W04	src/from_scratch/mlp.py	init_params(sizes, seed), forward(params, X) -> (P, cache), loss(params, X, Y), backward(params, cache, Y) -> grads; softmax output, mean cross-entropy	Rows of P sum to 1; every gradient agrees with central finite differences (relative error below 1e-5) for one and two hidden layers

Component	Required from	File	Interface	What the contract test checks
Naive Bayes (Ch. 4)	W05	src/from_scratch/naive_bayes.py	GaussianNB(var_smoothing) or MultinomialNB(alpha), at least one: fit, predict_log_proba, predict, classes_	Normalised log-posteriors and predictions agree with scikit-learn on synthetic data
HMM core (Ch. 6)	W09	src/from_scratch/hmm.py	forward(pi, A, B, obs) -> (alpha, loglik) or viterbi(pi, A, B, obs) -> (path, logprob), at least one	Likelihood, filtered marginals and best path agree with enumeration of all state paths; no underflow for T = 1000
PCA (Ch. 8)	W12	src/from_scratch/pca.py	fit_pca(X, k) -> (mean, components, explained_variance), transform, inverse_transform	Components equal the SVD directions up to sign; variances with ddof = 1; decorrelated scores; reconstruction error equals the discarded variance
Logistic / softmax regression (Ch. 10)	W15	src/from_scratch/logistic.py	softmax(Z), softmax_loss_grad(W, b, X, y, reg) -> (loss, dW, db), class SoftmaxRegression	No overflow for logits of size 1000; exact gradients against finite differences, with and without L2; separable blobs fitted above 95% accuracy

- Keep the file names, function names and signatures. You may add helper functions and your own tests in tests/.

- Inside these files use NumPy and scipy.special only. No scikit-learn or other library estimator may implement the required mechanism; scikit-learn belongs in the benchmark code.
- Passing the contract tests shows that the mechanism computes the right quantities on small cases. It does not show that you wrote it, and it does not replace the derivation, the experiment or the ownership check.
- Do not edit tests/contract, tools/ or .github/. The instructor compares these files with the released version, and a changed copy makes the automatic result unusable.

End of assignment specification. The purpose of the weekly structure is to make learning visible early enough to correct misconceptions before the final exam—not merely to audit work after the semester is over.